

RAG — Retrieval-Augmented Generation

Complete beginner guide: Chunking · Embedding · Retrieval · Evaluation · Tools

01 What is RAG and Why Does It Exist?

LLMs like GPT-4 are trained on data up to a certain date and cannot access your private documents or the latest information. **RAG (Retrieval-Augmented Generation)** solves this by fetching relevant information from your own knowledge base at query time and injecting it into the model's context before it answers.

Simple analogy: Imagine asking an expert a question, but before they answer, their assistant hands them the exact pages from the right book. The expert (LLM) now answers using that fresh, relevant material — not just memory.

CORE IDEA

RAG = Retrieve the right context → Augment the prompt with it → Generate a grounded answer.

The RAG Pipeline — 6 Stages

Stage	What Happens	Input	Output
1. Document Loading	Read raw files into memory	PDF, HTML, JSON, DB	Raw text
2. Chunking	Split text into smaller pieces	Long raw text	List of chunks
3. Embedding	Convert chunks to number vectors	Text chunks	Vectors (float arrays)
4. Indexing	Store vectors in a searchable DB	Vectors + metadata	Vector index
5. Retrieval	Find top-K chunks for a query	Query + index	Top-K chunks
6. Generation	LLM answers using retrieved chunks	Query + chunks	Final answer

02 Stage 1 — Document Loading

Before any RAG processing, you must load your documents from wherever they live. Each source type needs a different loader.

Source Type	Tool / Library	Notes
-------------	----------------	-------

PDF (text-based)	PyMuPDF, pdfplumber, LangChain PyPDFLoader	Preserves layout; extract tables separately
PDF (scanned/image)	AWS Textract, Tesseract OCR, Azure Form Recognizer	OCR required; quality depends on scan
HTML / Web page	BeautifulSoup, Playwright, LangChain WebBaseLoader	Strip nav/footer; JS pages need headless browser
Word / DOCX	python-docx, LangChain Docx2txtLoader	Preserves headings as metadata
CSV / Excel	pandas, LangChain CSVLoader	Each row becomes a chunk candidate
JSON / REST API	requests + json.loads, LangChain JSONLoader	Use jq-path to target nested fields
Database (SQL)	SQLAlchemy, LangChain SQLDatabaseLoader	Query → rows → serialize as text
Code files	LangChain GenericLoader + language parser	Use AST-aware splitting after loading
Audio / Video	Whisper (transcription) → then text pipeline	Add speaker + timestamp metadata

```
# LangChain – Load multiple source types
from langchain_community.document_loaders import (
    PyPDFLoader, WebBaseLoader, CSVLoader)

pdf_docs = PyPDFLoader("report.pdf").load()
web_docs = WebBaseLoader("https://docs.example.com").load()
csv_docs = CSVLoader("data.csv", source_column="text").load()

all_docs = pdf_docs + web_docs + csv_docs
# Each doc has: doc.page_content (text) + doc.metadata (source, page...)
```

03 Stage 2 — Chunking

Chunking splits long documents into smaller pieces. Each chunk becomes one unit that gets embedded and retrieved. **Too small** = chunks lose context. **Too large** = chunks are noisy and waste token budget.

Method 1: Fixed-Size / Sliding Window

Split every N tokens, with overlap of M tokens between adjacent chunks so no sentence is cut mid-thought.

BEST FOR

Plain text, chat logs, API responses, any unstructured continuous prose.

WATCH OUT

Can slice through paragraphs. Always use overlap (10–15% of chunk size).

```
# LangChain – RecursiveCharacterTextSplitter
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,          # target token size
    chunk_overlap=64,        # 64-token overlap between chunks
    separators=["\n\n", "\n", ". ", " ", ""]
    # tries paragraph break first, then sentence, then word
)

chunks = splitter.split_documents(all_docs)
# Each chunk: chunk.page_content + chunk.metadata
```

Method 2: Structure-Aware (Header-Based)

Use document structure — H1/H2/H3 headers in Markdown/HTML, or detected headings in PDFs — as chunk boundaries. Each chunk stores its heading path as metadata.

BEST FOR

PDFs with sections, technical documentation, books, legal documents. Store heading hierarchy as metadata for downstream filtering.

```
# LangChain – MarkdownHeaderTextSplitter
from langchain.text_splitter import MarkdownHeaderTextSplitter

headers = [("#", "H1"), ("##", "H2"), ("###", "H3")]
splitter = MarkdownHeaderTextSplitter(headers_to_split_on=headers)
chunks = splitter.split_text(markdown_text)
# chunk.metadata = {"H1": "Chapter 3", "H2": "Section 2.1"}
```

Method 3: Semantic Chunking

Embed every sentence. Where the cosine similarity between adjacent sentences drops sharply, place a chunk boundary. Each chunk contains one coherent topic.

BEST FOR

Research papers, reports that shift topics frequently. Higher quality but more expensive — requires an embedding pass during indexing.

Method 4: Agentic / Proposition Chunking

An LLM reads each passage and decides whether the next sentence belongs to the current chunk or starts a new one. Highest quality, highest cost. Use for premium knowledge bases.

Chunking by Data Type — Quick Reference

Data Type	Best Method	Chunk Size	Key Note
-----------	-------------	------------	----------

Plain text / articles	Recursive char split	512–1024 tok	Use overlap
PDF (structured)	Header-aware + recursive	512–1024 tok	Store section path as metadata
PDF (scanned)	OCR → recursive char split	512 tok	Clean OCR noise first
Tables (PDF/CSV)	Row-level or table-as-chunk	One row or table	Prepend column headers to each row
Images	Vision LLM caption → embed caption	N/A	Store image path with caption chunk
HTML pages	DOM parse → strip boilerplate → recursive	512 tok	Preserve h1–h4 as metadata
JSON / API	Record-level chunking	One record	Serialize as "field: value" pairs
Code	AST-based (function/class boundary)	One function	Store file path + class context
Audio transcript	Speaker-diarized + sentence split	30s window	Store speaker + timestamp

Advanced: Parent-Document Retrieval

Index small child chunks (128 tokens) for precise matching. When a child chunk is retrieved, return its parent chunk (512+ tokens) to the LLM for richer context. Best of both worlds: precision in retrieval, context in generation.

04 Stage 3 — Embedding

An **embedding** converts text into a list of floating-point numbers (a vector). Semantically similar texts produce vectors that are "close" in vector space. This is what allows us to search by meaning, not just by keywords.

INTUITION

"cat" and "kitten" produce vectors that are close together. "cat" and "database" produce vectors far apart. The distance between vectors = semantic distance between meanings.

Family 1: Sparse Embeddings (Keyword-Based)

Most vector dimensions are zero. Non-zero values represent importance of specific words. Very interpretable. Excel at exact term matching.

TF-IDF — Term Frequency × Inverse Document Frequency

Each vector dimension = one vocabulary word. The value = how "important" that word is to this document. Words common across all documents (the, is, a) get low scores. Rare, document-specific words get high scores.

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t) = (\text{count of } t \text{ in } d / \text{total words in } d) \times \log(N / \text{df}(t))$$

N = total docs in corpus. $\text{df}(t)$ = number of docs containing term t .

WORKED EXAMPLE

Corpus: 1000 docs. Query: "neural network". "neural" appears 5 times in doc, doc has 200 words, appears in 10 of 1000 docs. $TF = 5/200 = 0.025$. $IDF = \log(1000/10) = 2.0$. $TF-IDF = 0.025 \times 2.0 = 0.05$ "the" appears 20 times but in all 1000 docs: $IDF = \log(1000/1000) = 0$. Score = 0.

BEST USE CASES

Exact product codes, part numbers, legal/medical terms, error codes — any domain where vocabulary match = meaning match.

LIMITATION

No synonyms. "car" and "automobile" score zero similarity. No word order or semantic understanding.

BM25 — Best Match 25 (Better Baseline)

BM25 improves TF-IDF with two fixes: (1) **TF saturation** — a word appearing 50 times isn't 50× more important than appearing once; (2) **Length normalization** — a word in a short doc is more significant than in a 10,000-word doc.

$$BM25(t, d) = IDF(t) \times [TF(t, d) \times (k1+1)] / [TF(t, d) + k1 \times (1 - b + b \times |d|/avgdl)]$$

$k1 = 1.2-2.0$ (TF saturation). $b = 0.75$ (length normalization). $avgdl$ = avg doc length in corpus.

INDUSTRY STANDARD

BM25 powers Elasticsearch, OpenSearch, and Solr. The default for production keyword search. Always prefer over plain TF-IDF.

Family 2: Dense Embeddings (Neural / Semantic)

Transformer neural networks encode text into dense vectors where ALL dimensions have values. The model has learned semantic relationships — synonyms, paraphrases, and cross-lingual equivalences all produce nearby vectors.

Model	Dims	Speed	Best For
all-MiniLM-L6-v2	384	Fast (local)	General Q&A;,, prototyping, local dev. Great baseline.
all-mpnet-base-v2	768	Medium (local)	Higher quality general semantic search. Better than MiniLM.
BGE-M3	1024	Medium (local)	Multi-lingual, supports dense+sparse+colbert. SOTA open model.
E5-large-instruct	1024	Medium (local)	Prefix "query:" / "passage:" for best results. Strong performance.

text-embedding-3-small	1536	Fast (API)	OpenAI. Good multilingual. Matryoshka — truncatable.
text-embedding-3-large	3072	Medium (API)	OpenAI. Highest accuracy. Use when quality > cost.
Cohere embed-v3	1024	Fast (API)	int8/binary quantization. Great for large-scale production.

```
# HuggingFace sentence-transformers – embed chunks
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")

texts = [chunk.page_content for chunk in chunks]
embeddings = model.encode(texts, batch_size=64, show_progress_bar=True)
# embeddings.shape = (num_chunks, 384)
# embeddings[i] is the vector for texts[i]
```

Family 3: Hybrid (Sparse + Dense Together)

Run both BM25 (keyword) and dense embedding simultaneously. Combine ranked results using **Reciprocal Rank Fusion (RRF)**. This is the production default — handles both exact keyword queries AND semantic queries robustly.

RRF_score(doc) = SUM [1 / (k + rank(doc, list_i))] where k = 60

For each ranking list i, get the rank of the document. Higher rank = smaller denominator = higher score.

RRF EXAMPLE

Query: "Python async error handling" Semantic top-3: [DocA(1), DocB(2), DocC(3)] Keyword top-3: [DocB(1), DocD(2), DocA(3)] DocA RRF = $1/(60+1) + 1/(60+3) = 0.01639 + 0.01563 = 0.03202$ DocB RRF = $1/(60+2) + 1/(60+1) = 0.01613 + 0.01639 = 0.03252$ Merged rank: DocB (1st), DocA (2nd), DocD (3rd), DocC (4th) DocB wins because it appears high in BOTH lists.

05 Stage 4 — Similarity Metrics

Once you have vectors, you need a way to measure how "close" a query vector is to document vectors. Three metrics matter. These are used at **retrieval time**, not during embedding creation.

Cosine Similarity — Direction Only

Measures the angle between two vectors, ignoring their length (magnitude). A short tweet and a long article about the same topic can have high cosine similarity because they point in the same semantic direction.

cosine(A, B) = (A · B) / (||A|| × ||B||) range: -1 to +1

For text embeddings, scores are typically 0.0 to 1.0. Higher = more similar.

WORKED EXAMPLE

Vector A (cat) = [0.9, 0.1, 0.0] Vector B (kitten) = [0.8, 0.2, 0.0] Dot product = (0.9×0.8) + (0.1×0.2) + (0×0) = 0.72 + 0.02 = 0.74 $\|A\| = \sqrt{0.81+0.01} = 0.906$ $\|B\| = \sqrt{0.64+0.04} = 0.825$ cosine = 0.74 / (0.906 × 0.825) = 0.74 / 0.748 = 0.989 (very similar!)

USE WHEN

Default for ALL text embedding RAG systems. Removes length bias. Safe for sentence-transformers, TF-IDF, BM25 sparse vectors.

Dot Product — Direction + Magnitude

$$\text{dot}(A, B) = \text{SUM}(A_i \times B_i) \text{ range: } -\text{inf to } +\text{inf}$$

When vectors are L2-normalized (unit vectors), dot product = cosine similarity.

USE WHEN

Vectors are known to be L2-normalized. OpenAI and Cohere normalize their embeddings — use dot product for speed (same result as cosine, but faster).

WARNING

On non-normalized vectors, high-magnitude vectors dominate regardless of semantic similarity. Only use if you know vectors are normalized.

Euclidean Distance — Straight-Line Distance

$$L2(A, B) = \sqrt{\text{SUM}(A_i - B_i)^2} \text{ range: } 0 \text{ to } +\text{inf}$$

Lower = more similar (opposite of cosine). 0 = identical vectors.

USE WHEN

Clustering tasks (k-means). Visualization (UMAP, t-SNE). Some specialized embedding models trained with L2 loss.

NEVER FOR RAG RETRIEVAL

Euclidean distance suffers from the "curse of dimensionality" at 300+ dimensions. All pairwise distances cluster around similar values — meaningless for ranking. Use cosine instead.

Metric	Use For	Avoid For	Default?
Cosine Similarity	All text RAG, TF-IDF, BM25, sentence-transformers	—	YES — safe default

Dot Product	OpenAI, Cohere embeddings (they normalize)	Non-normalized vectors	If API-normalized
Euclidean (L2)	Clustering, k-means, image embeddings	High-dim text RAG retrieval	NO

06 Stage 5 — Retrieval Algorithms

With vectors indexed, retrieval answers: "Given this query vector, find the top-K most similar document vectors." The challenge at scale: comparing against millions of vectors must be fast.

Exact Search — kNN (k-Nearest Neighbors)

Compare query vector against every single vector. Guaranteed to find the true top-K. $O(n \times d)$ per query — scales poorly. Suitable for small corpora or prototyping.

USE WHEN

Under 100K chunks, or when building/testing your pipeline. FAISS IndexFlatL2 / IndexFlatIP implement exact kNN.

Approximate Nearest Neighbors (ANN)

ANN algorithms trade a tiny accuracy loss (usually 1–5%) for 10–1000x speed gains. They build an index structure at "index time" that allows skipping most vectors during search.

HNSW — Hierarchical Navigable Small World

Builds a multi-layer graph. Top layers = sparse, long-range connections for fast coarse navigation. Bottom layers = dense, short-range connections for precise local search. Query traverses from top layer down, like zooming into a map.

WHY HNSW DOMINATES PRODUCTION

$O(\log n)$ query time. 95-99% recall. No training data needed. Used by default in Qdrant, Weaviate, Milvus, pgvector. Key parameters: M (connections per node, 16-64), ef_construction (build quality), ef_search (query accuracy, tunable at runtime).

LIMITATION

All vectors must fit in RAM. Expensive to update (deletions require rebuild). For >50M vectors, consider IVF-PQ or DiskANN.

IVF — Inverted File Index

Cluster all vectors into N "cells" using k-means at build time. At query time, only search vectors in the nearest few cells. nprobe controls how many cells to search: higher nprobe = more accurate but slower.

NPROBE TUNING

nprobe=1: fastest, least accurate. nprobe=nlist: equivalent to exact search. Typical: nprobe=10-20 for 1-5% accuracy loss and 10-50x speedup.

IVF-PQ — IVF + Product Quantization

Compresses each vector into a short code (16-64x compression). Massive memory savings — enables billion-scale search on a single machine. Used via FAISS IndexIVFPQ.

LSH — Locality Sensitive Hashing

Hash similar vectors into the same bucket. At query time, only compare with bucket members. Simple, but lower recall than HNSW. Best for sparse (TF-IDF/BM25) vectors or memory-constrained environments.

DiskANN / ScaNN

DiskANN (Microsoft): stores graph on disk instead of RAM. Billion-scale ANN with minimal memory. Used in Azure AI Search. **ScaNN** (Google): highly optimized ANN with anisotropic quantization. Used in Google products.

Algorithm	Scale	Recall	Memory	Best For
Exact kNN	<100K	100%	Low	Prototypes, small corpora
HNSW	<50M	95-99%	High (RAM)	Production RAG — DEFAULT
IVF-Flat	Mid-scale	90-98%	Medium	Memory-constrained production
IVF-PQ	Billions	85-95%	Very low	Massive scale, compressed
LSH	Any	70-90%	Low	Sparse vectors, simple cases
DiskANN	Billions	95%+	Minimal RAM	Billion+ vectors, cloud

Vector Databases — Where Your Index Lives

Database	Index	Hybrid?	Best For
Qdrant	HNSW	Yes (SPLADE/BM42)	Production hybrid search, Rust-based, fast, open source
Weaviate	HNSW	Yes (BM25)	GraphQL API, multi-modal, managed cloud option
Milvus / Zilliz	HNSW / IVF-PQ	Limited	Massive scale, cloud-native, enterprise
Chroma	HNSW	No	Local dev, prototyping — easiest to get started
FAISS	All types	No	Research, custom pipelines, offline batch indexing
pgvector	HNSW / IVF	No	Already using PostgreSQL — simplest stack addition
Pinecone	Proprietary	Yes	Managed serverless, no ops, fast production deployment

```
# Qdrant — create collection and upsert vectors
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams, PointStruct

client = QdrantClient(":memory:") # or host="localhost", port=6333
client.create_collection("my_docs",
    vectors_config=VectorParams(size=384, distance=Distance.COSINE))
points = [PointStruct(id=i, vector=embeddings[i].tolist()),
```

```

        payload={"text": texts[i], "source": "doc.pdf"})
    for i in range(len(texts))]

client.upsert(collection_name="my_docs", points=points)

# Search
results = client.search("my_docs", query_vector=query_embedding,
                        limit=5, with_payload=True)

```

07 Stage 6 — Generation (RAG Prompt + LLM)

After retrieving the top-K chunks, you construct a prompt that includes the query and the retrieved context, then pass it to the LLM. The LLM must answer using only the provided context.

```

# RAG prompt template
SYSTEM_PROMPT = """
You are a helpful assistant. Answer the user's question using ONLY
the context provided below. If the answer is not in the context,
say "I don't have enough information to answer this."
"""

def build_rag_prompt(query, retrieved_chunks):
    context = "\n\n".join(
        f"[{i+1}] {c.page_content}" for i, c in enumerate(retrieved_chunks)
    )
    return f"""
Context:
{context}

Question: {query}

Answer: """

```

```

# Full RAG pipeline – end to end
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.vectorstores import Qdrant
from langchain.chains import RetrievalQA

# 1. Embed and index (done once)
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vectorstore = Qdrant.from_documents(chunks, embeddings,
                                   location=":memory:", collection_name="docs")

# 2. Create retriever
retriever = vectorstore.as_retriever(search_kwargs={"k": 5})

# 3. Create RAG chain
llm = ChatOpenAI(model="gpt-4o-mini")
rag_chain = RetrievalQA.from_chain_type(llm=llm,
                                       retriever=retriever, return_source_documents=True)

```

```
# 4. Query
result = rag_chain.invoke({"query": "What is the refund policy?"})
print(result["result"]) # The answer
print(result["source_documents"]) # Which chunks were used
```

08 Stage 7 — Evaluation

Evaluation answers: "Is my RAG system actually working?" You must evaluate **retrieval** (did I fetch the right chunks?) and **generation** (did the LLM produce a correct, grounded answer?) separately.

WHY SEPARATE?

Perfect retriever + bad generator = wrong answers. Perfect generator + bad retriever = hallucinations. Only end-to-end evaluation hides which component is failing.

Retrieval Metrics (with worked examples)

Setup for all examples: Corpus = docs D1–D10. Ground truth relevant docs for query Q = {D1, D3, D5, D7}. Your retriever returns top-5: [D1, D2, D3, D6, D7].

Precision@K — Quality of Results

Of the K docs you returned, what fraction were actually relevant?

$$\text{Precision@K} = |\text{Retrieved_top_K} \cap \text{Relevant}| / K$$

EXAMPLE

Retrieved top-5: {D1, D2, D3, D6, D7}. Relevant: {D1, D3, D5, D7} Intersection: {D1, D3, D7} = 3 hits. K = 5 Precision@5 = 3/5 = 0.60 (60% of what we returned was good)

Recall@K — Coverage

Of ALL relevant docs in the corpus, what fraction did you find?

$$\text{Recall@K} = |\text{Retrieved_top_K} \cap \text{Relevant}| / |\text{Relevant}|$$

EXAMPLE

Intersection: {D1, D3, D7} = 3. Total relevant = 4 (D1,D3,D5,D7) Recall@5 = 3/4 = 0.75 (we missed D5)

F1@K — Harmonic Balance of P and R

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

EXAMPLE

$$F1 = 2 \times (0.60 \times 0.75) / (0.60 + 0.75) = 1.2 / 1.35 = 0.667$$

MRR — Mean Reciprocal Rank

Where does the FIRST relevant doc appear? Ideal for Q&A; where you need ONE correct answer.

$$RR = 1 / \text{rank_of_first_relevant_doc}. \text{ MRR} = \text{mean}(RR) \text{ over all queries}$$

EXAMPLE

Ranking: [D1✓, D2✗, D3✓, D6✗, D7✓]. First relevant = D1 at rank 1. $RR = 1/1 = 1.00$

MAP — Mean Average Precision

AP rewards relevant docs appearing early in the ranking. MAP = mean AP over all queries.

$$AP = (1/|\text{Relevant}|) \times \text{SUM}[P@k \times \text{rel}(k)] \text{ (sum over positions where relevant doc appears)}$$

EXAMPLE

Ranking: [D1✓, D2✗, D3✓, D6✗, D7✓]. $|\text{Relevant}|=4$ $k=1$: D1 relevant $\rightarrow P@1=1.00$ | $k=3$: D3 relevant $\rightarrow P@3=0.667$ | $k=5$: D7 relevant $\rightarrow P@5=0.60$ | D5 missing $\rightarrow 0$ $AP = (1/4) \times (1.00 + 0.667 + 0.60 + 0) = 2.267/4 = 0.567$

NDCG@K — Normalized Discounted Cumulative Gain

The gold standard. Handles GRADED relevance (very relevant=3, somewhat=1, irrelevant=0) and discounts lower-ranked positions.

$$DCG@K = \text{SUM}(k=1 \text{ to } K) [\text{rel}(k) / \log_2(k+1)] \text{ NDCG} = DCG / IDC G$$

IDCG = DCG of the ideal (perfect) ranking. NDCG = 1.0 only if your ranking is perfect.

EXAMPLE

Graded scores: D1=3, D2=0, D3=2, D5=3, D6=0, D7=1. Ranking: [D1,D2,D3,D6,D7] $DCG@5 = 3/\log_2(2) + 0/\log_2(3) + 2/\log_2(4) + 0/\log_2(5) + 1/\log_2(6) = 3/1.0 + 0 + 2/2.0 + 0 + 1/2.585 = 3.0 + 1.0 + 0.387 = 4.387$ Ideal order: [D1(3),D5(3),D3(2),D7(1),D2(0)] $IDCG@5 = 3/1 + 3/1.585 + 2/2 + 1/2.322 + 0 = 3 + 1.893 + 1 + 0.431 = 6.324$ $NDCG@5 = 4.387 / 6.324 = 0.694$ (69.4% of ideal)

Generation Metrics

Metric	What It Measures	Formula / Method
Faithfulness	Are ALL answer claims supported by context? (Anti-hallucination)	$ \text{Supported claims} / \text{Total claims in answer} $

Answer Relevance	Does the answer address the question asked?	$\text{cosine_sim}(\text{embed}(\text{reconstructed_questions}), \text{embed}(\text{original_query}))$
Context Precision	Are retrieved chunks useful, or mostly noise?	$ \text{Chunks used in answer} / \text{Total chunks retrieved} $
Context Recall	Did context contain all information needed?	$ \text{Answer claims in context} / \text{Claims in reference answer} $
ROUGE-1 F1	Word overlap with reference answer	$2 \times (P \times R) / (P + R)$ where P,R = unigram overlap fractions
BERTScore	Semantic similarity to reference (better than ROUGE)	Max cosine sim between BERT token embeddings

RAGAS — THE FRAMEWORK

RAGAS (pip install ragas) combines Faithfulness + Answer Relevance + Context Precision + Context Recall into one evaluation suite. Uses LLM-as-judge — no manual labeling required. The industry standard for RAG evaluation.

```
# RAGAS evaluation
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy,
                        context_precision, context_recall
from datasets import Dataset

data = {
    "question": ["What is the refund policy?"],
    "answer": ["Refunds are processed within 30 days."],
    "contexts": [["Our refund policy allows returns within 30 days..."]],
    "ground_truth": ["Refunds take 30 days to process."]
}

result = evaluate(Dataset.from_dict(data),
                  metrics=[faithfulness, answer_relevancy, context_precision, context_recall])
print(result)
```

Diagnostic Guide — What to Fix When

Symptom	Root Cause	Fix
Low Faithfulness	LLM ignores context or adds unsupported facts	Stronger system prompt, better model, reduce top-K
Low Context Recall	Retriever misses relevant chunks	Fix chunking (too large?), improve embedding model, try hybrid search
Low Context Precision	Retriever returns too many irrelevant chunks	Add re-ranking, reduce K, add metadata filters
Low Answer Relevance	LLM goes off-topic	Improve system prompt, instruction-tune the model

Low MRR	Correct answer is ranked too low	Better embedding model, hybrid search, re-ranker
Low NDCG but high P@1	First result good, rest poor	Improve diversity in retrieval, MMR retrieval

09 RAG Tools Ecosystem — Complete Reference

Category	Tool	Use It For
Framework	LangChain	End-to-end RAG chains, loaders, splitters, retrievers. Most popular.
Framework	LlamaIndex	Document-centric RAG, richer indexing options, better for structured data.
Framework	Haystack (deepset)	Production pipelines, strong evaluation, modular architecture.
Embedding (local)	sentence-transformers	Run any HuggingFace embedding model locally. pip install sentence-transformers.
Embedding (API)	OpenAI Embeddings API	text-embedding-3-small / large. Managed, high quality.
Embedding (API)	Cohere Embed v3	Multilingual, int8/binary quantization support.
Vector DB (local)	Chroma	Easiest local vector DB. pip install chromadb. Great for prototyping.
Vector DB (local)	FAISS	Facebook AI library. In-memory or on-disk. Most flexible for custom indexing.
Vector DB (prod)	Qdrant	Open source + cloud. HNSW + hybrid search. Best OSS production choice.
Vector DB (prod)	Weaviate	Multi-modal, GraphQL, managed cloud. Strong ecosystem.
Vector DB (SaaS)	Pinecone	Fully managed, serverless, easiest to scale. No infra to manage.
Reranking	Cohere Rerank	Cross-encoder reranking API. Retrieve 50, rerank to top 5.
Reranking	FlashRank / BGE Reranker	Local cross-encoder reranking. Free.
Evaluation	RAGAS	pip install ragas. Faithfulness, relevance, context metrics. LLM-as-judge.
Evaluation	TruLens	Full RAG evaluation + tracing + dashboard. Alternative to RAGAS.
LLM (API)	OpenAI GPT-4o-mini	Best cost/quality ratio for RAG generation.
LLM (local)	Ollama + Llama 3 / Mistral	Run LLMs locally. No API cost. pip install ollama.
Observability	LangSmith	Trace every RAG call. Debug retrieval + generation.
Observability	Arize Phoenix	Open source RAG tracing and evaluation dashboard.

WHAT TO BUILD NEXT

1. Install: `pip install langchain chromadb sentence-transformers ragas` 2. Build a basic RAG with Chroma + all-MiniLM-L6-v2 on a PDF you own 3. Evaluate with RAGAS 4. Switch to Qdrant + hybrid search and compare NDCG scores 5. Add a cross-encoder reranker and measure improvement